Cosmos EVM 프리컴파일 시스템

1. 프리컴파일 개요

1.1 정의

프리컴파일(Precompile)은 EVM에서 Cosmos SDK의 네이티브 기능에 접근할 수 있게 해주는 특별한 스마트 컨트랙트입니다.

1.2 작동 원리

- **특정 주소**: 각 프리컴파일은 고정된 이더리움 주소를 가짐
- **네이티브 실행**: Go 코드로 직접 구현되어 높은 성능
- **Solidity 인터페이스**: 표준 스마트 컨트랙트처럼 호출 가능

1.3 주소 할당

```
// 프리컴파일 주소 범위: 0x0000000000000000000000000000000000000400-0x000000000000
const (
BankAddress = "0x0000000000000000000000000000000000000400"
StakingAddress = "0x0000000000000000000000000000000000000401"
DistributionAddress = "0x0000000000000000000000000000000000000402"
// ... 기타 프리컴파일 주소들
)
```

2. 사용 가능한 프리컴파일

2.1 Bank 프리컴파일

주소: `0x0000000000000000000000000000000000000400`

기능

- 토큰 잔액 조회
- 토큰 전송
- 토큰 메타데이터 조회

Solidity 인터페이스

```
interface IBank {
function balanceOf(address account, string memory denom)
external view returns (uint256);
function transfer(address to, string memory denom, uint256 amount)
external returns (bool);
function totalSupply(string memory denom)
external view returns (uint256);
}
```

2.2 Staking 프리컴파일

주소: `0x0000000000000000000000000000000000000401`

기능

- 밸리데이터 정보 조회
- 위임(Delegation) 관리
- 언본딩 처리
- 리워드 조회

Solidity 인터페이스

```
interface IStaking {
function delegate(string memory validatorAddr, uint256 amount)
external returns (bool);
function undelegate(string memory validatorAddr, uint256 amount)
external returns (bool);
function getDelegation(address delegator, string memory validatorAddr)
external view returns (uint256);
}
```

2.3 Distribution 프리컴파일

주소: `0x0000000000000000000000000000000000000402`

기능

- 스테이킹 리워드 조회
- 리워드 인출
- 커미션 관리

Solidity 인터페이스

```
interface IDistribution {
function withdrawDelegatorReward(string memory validatorAddr)
external returns (bool);
function getDelegatorReward(address delegator, string memory validatorAddr)
external view returns (uint256);
}
```

2.4 ERC20 프리컴파일

주소: `0x0000000000000000000000000000000000000403`

기능

- Cosmos 코인과 ERC20 토큰 간 변환
- 토큰 페어 관리
- 변환 이벤트 처리

2.5 ICS20 프리컴파일

주소: `0x0000000000000000000000000000000000000404`

기능

- IBC 토큰 전송
- 크로스체인 통신
- 채널 관리

2.6 기타 프리컴파일

- **Gov**: 거버넌스 제안 및 투표
- **Slashing**: 슬래싱 정보 조회
- **Bech32**: 주소 형식 변환
- **P256**: secp256r1 서명 검증

3. 프리컴파일 구현 구조

3.1 기본 인터페이스

```
type StaticPrecompiledContract interface {
// 프리컴파일 주소 반환
Address() common.Address
// 가스 비용 계산
RequiredGas(input []byte) uint64
// 실제 실행 로직
Run(evm *vm.EVM, contract *vm.Contract, readonly  bool) ([]byte, error)
}
```

3.2 구현 예시 (Bank 프리컴파일)

```
type BankPrecompile struct {
bankKeeper bankkeeper.Keeper
accountKeeper authkeeper.AccountKeeper
}
```

```go
func (p *BankPrecompile) Run(evm *vm.EVM, contract *vm.Contract, readonly bool)
// 입력 데이터 파싱
method, args, err := p.parseInput(contract.Input)
if err != nil {
return nil, err
}
// 메서드별 처리
switch method.Name {
case "balanceOf":
return p.balanceOf(evm, args)
case "transfer":
return p.transfer(evm, args, readonly)
default:
return nil, fmt.Errorf("unknown method: %s", method.Name)
}
}
```

4. 프리컴파일 등록 및 관리

4.1 등록 과정

```go
func NewAvailableStaticPrecompiles(
```

```
stakingKeeper stakingkeeper.Keeper,

distributionKeeper distrkeeper.Keeper,

bankKeeper bankkeeper.Keeper,

// ... 기타 키퍼들

) map[common.Address]vm.PrecompiledContract {

precompiles := make(map[common.Address]vm.PrecompiledContract)

// Bank 프리컴파일 등록

bankPrecompile := bank.NewPrecompile(bankKeeper, accountKeeper)

precompiles[bankPrecompile.Address()] = bankPrecompile

// Staking 프리컴파일 등록

stakingPrecompile := staking.NewPrecompile(stakingKeeper, authzKeeper)

precompiles[stakingPrecompile.Address()] = stakingPrecompile

// ... 기타 프리컴파일들 등록

return  precompiles

}
```

4.2 EVM 통합

```
// EVM 키퍼에서 프리컴파일 설정

func (k *Keeper) WithStaticPrecompiles(precompiles  map[common.Address]vm.Precom

k.precompiles = precompiles

}
```

5. 사용 예시

5.1 Solidity에서 Bank 프리컴파일 사용

```
pragma solidity ^0.8.0;


contract CosmosIntegration {
address constant BANK_PRECOMPILE = 0x0000000000000000000000000000000000000400;
function getBalance(address account, string memory denom)
public view returns (uint256) {
(bool success, bytes memory data) = BANK_PRECOMPILE.staticcall(
abi.encodeWithSignature("balanceOf(address,string)", account, denom)
);
require(success, "Bank precompile call failed");
return abi.decode(data, (uint256));
}
function transferTokens(address to, string memory denom, uint256 amount)
public returns (bool) {
(bool success, bytes memory data) = BANK_PRECOMPILE.call(
abi.encodeWithSignature("transfer(address,string,uint256)", to, denom, amount)
);
require(success, "Transfer failed");
return abi.decode(data, (bool));
```

```
}

}
```

5.2 JavaScript에서 프리컴파일 호출

```
const bankPrecompileAddress = "0x0000000000000000000000000000000000000400";

const bankABI = [

"function balanceOf(address account, string denom) view returns (uint256)",

"function transfer(address to, string denom, uint256 amount) returns (bool)"

];


const bankContract = new ethers.Contract(bankPrecompileAddress, bankABI, p


// 잔액 조회

const balance = await bankContract.balanceOf(userAddress, "atest");


// 토큰 전송

const tx = await bankContract.transfer(recipientAddress, "atest", ethers.p
```

6. 커스텀 프리컴파일 개발

6.1 개발 단계

1. **인터페이스 정의**: Solidity ABI 설계

2. **Go 구현**: 프리컴파일 로직 작성

3. **테스트**: 단위 테스트 및 통합 테스트

4. **등록**: EVM 키퍼에 프리컴파일 등록

6.2 모범 사례

- **가스 효율성**: 적절한 가스 비용 설정
- **에러 처리**: 명확한 에러 메시지 제공
- **보안**: 입력 검증 및 권한 확인
- **호환성**: 이더리움 표준과의 일관성 유지

이 문서는 Cosmos EVM의 프리컴파일 시스템을 상세히 설명합니다. 다음 문서에서는 트랜잭션 처리 과정을 살펴보겠습니다.